

AUTONOMOUS MARINE RESEARCH FRAMEWORK

Complete Project Study Report - From Scratch to Final Demonstration

ROS 2 Humble + Gazebo Harmonic + Guidance + PID/SMC + YOLO + Mission Autonomy + Cooperative
USVs

*Prepared as a learning and viva reference covering the complete implementation history, experiments, failures,
corrections, validation, visualization work, and final demo workflow.*

Workspace used throughout the completed framework: /home/sireesha/marine_ws

How to Use This Report

This document is intentionally written as both a project history and a study guide. Read Chapters 1-4 first if you are starting from zero. Then study the phase chapters in order. Each phase explains the objective, what was implemented, why it was needed, the important ROS 2/Gazebo concepts, tests performed, failures encountered, and what was frozen at the end of that phase.

The report preserves the project's actual scope and limitations. Where a result was a simulation approximation rather than a real-world physical model, that distinction is stated explicitly. Where an experiment failed, the failure is retained rather than rewritten as a success.

Contents

1. Project objective and final system
2. Background concepts to learn first
3. Software environment and workspace setup
4. Overall architecture and ROS 2 data flow
5. Phase 1 - Calm ocean, buoyancy and first Gazebo scene
6. Phase 2 - Physical catamaran, thrusters, odometry, IMU and TF
7. Phase 3 - Cubic-spline path generation and LOS guidance
8. Phase 4 - PID path-following controller
9. Phase 5 - Camera, YOLO perception, search, approach and inspection mission
10. Phase 6 - Sliding Mode Control (SMC)
11. Phase 7 - Deterministic disturbance and robustness testing
12. Phase 8 - Two-USV cooperative formation control
13. Phase 9 - Final integration, traceability and reproducibility
14. Final demo GUI, RViz, HUD and wavy-water presentation suite
15. Build/runtime problems solved during demo preparation
16. Current commands for running all simulation modes
17. Future real gimbal-camera integration
18. Key limitations and scientific claims you should and should not make
19. From-scratch study plan

20. Viva questions and short answers

Appendix A. Important parameters and results

Appendix B. Important ROS 2 topics and frames

Appendix C. Troubleshooting checklist

Appendix D. Glossary

1. Project Objective and Final System

The project grew into a modular autonomous marine robotics framework rather than a single controller demo. The final objective was to simulate an under-actuated autonomous surface vehicle (ASV), generate smooth paths, follow those paths using interchangeable controllers, detect a vessel with a frozen YOLO model, execute autonomous mission logic, recover from temporary target loss, stop safely at a stand-off distance, and finally coordinate a second ASV in formation.

1.1 Final capability chain

Perception -> Detection confirmation -> Guidance -> Control -> Mission arbitration -> Safe arrival -> Inspection

control
|
+--> Cooperative state exchange -> Follower formation

A major design principle was separation of concerns. Perception never directly drove the thrusters. Guidance generated desired references. Controllers converted reference error into motion commands. Mission logic decided which behavior had authority. Safety logic could force zero propulsion. This separation made the system testable and allowed PID, SMC, mission behavior and cooperative control to be evaluated independently.

1.2 Final software stack

Layer	Technology / implementation
Operating system	Ubuntu 22.04.5 LTS under WSL2
Middleware	ROS 2 Humble
Simulator	Gazebo Harmonic / Gazebo Sim 8.x
ROS-Gazebo integration	ros_gz_bridge / ros_gzharmonic
Primary language	Python 3.10 plus C++ Gazebo plugin
Perception	Frozen YOLOv5n6/P6 checkpoint, 10 classes, generic-vessel mission behavior
Guidance	Natural cubic splines + Line-of-Sight (LOS)
Controllers	PID and nonlinear Sliding Mode Control
Mission autonomy	Deterministic finite-state logic with confirmation, target-loss recovery and safety arbitration
Cooperation	Leader-state publisher + follower Serret-Frenet formation logic
Visualization	Gazebo GUI, RViz2, OpenCV HUD, terminal telemetry HUD

1.3 What the final framework can demonstrate

- A catamaran-like ASV with buoyancy, drag, differential thrust, odometry and IMU.
- Straight, curved and S-shaped spline path following.
- Selectable PID or SMC heading control.
- Deterministic external-force and yaw disturbances.
- Thermal-style camera perception and frozen YOLO vessel detection.
- Coverage search, temporal detection confirmation, target loss and reacquisition.

- Visual-guided approach and independent depth/range-based stand-off arrival.
- Zero-propulsion safety before inspection.
- Two-ASV in-trail formation with communication-delay/dropout handling.
- Reproducible experiments, regression tests, metrics, evidence folders and managed teardown.

2. Background Concepts to Learn First

2.1 ROS 2

ROS 2 is the middleware connecting independent processes called nodes. Nodes publish messages to topics and subscribe to the topics they need. Services provide request/response behavior, parameters configure nodes, and TF represents coordinate-frame relationships. In this project, ROS 2 is the communication backbone between simulator sensors, guidance, controllers, mission logic, perception, and cooperative coordination.

2.2 Gazebo Harmonic

Gazebo Harmonic is the 3D physics simulator used here. It runs the world, ASV model, joints, thrusters, sensors and rendering. The project uses the modern Gazebo Sim stack (gz-sim 8), not Gazebo Classic and not a lite edition.

2.3 Under-actuated marine vehicle

The ASV is under-actuated because the two main control inputs are port and starboard thrust. There is no independent lateral thruster. Surge and yaw are directly influenced by differential thrust, while lateral motion is a result of vehicle dynamics and heading.

2.4 Guidance versus control

Guidance decides where the vehicle should go and what heading it should aim for. Control decides how strongly to actuate the vehicle to follow that reference. LOS guidance therefore sits above the PID/SMC controller.

2.5 Coordinate frames

You should understand world/odom, base_link and sensor frames. A correct TF tree lets the system transform positions and directions between the inertial/world frame and the moving vehicle/sensor frames. This becomes especially important for visual target bearing and gimbal integration.

2.6 Error metrics

- Cross-track error (CTE): perpendicular distance from vehicle to reference path.
- Heading error: wrapped angular difference between desired and actual heading.
- RMSE: square-root of the mean squared error across a run.
- Formation position error: follower difference from the desired leader-relative slot.
- Relative heading error: follower-versus-leader heading difference.

3. Software Environment and Workspace Setup

3.1 Frozen environment

Item	Value
Ubuntu	22.04.5 LTS (Jammy)
ROS 2	Humble
Gazebo	Harmonic / Gazebo Sim 8.15.x
Python	3.10.12
Workspace	/home/sireesha/marine_ws
Important rule	Do not modify the older /home/sireesha/maritime_ws workspace
GPU rule	Do not change NVIDIA drivers as part of this project

3.2 Typical shell preparation

```
cd /home/sireesha/marine_ws
source /opt/ros/humble/setup.bash
source install/local_setup.bash 2>/dev/null || true
source install-phase7/local_setup.bash
export LIBGL_ALWAYS_SOFTWARE=1
```

During final demo preparation the software-rendering environment variable was kept as a stability fallback because the Gazebo GUI renderer had previously crashed under the native WSLg rendering path. The simulation server itself was healthy; the crash was isolated to the GUI renderer.

3.3 Package-level modularity

The framework is split into packages such as `marine_description`, `marine_gazebo`, `marine_bringup`, `marine_control`, `marine_guidance`, perception/mission components, and a C++ disturbance plugin. This modularity was essential because each research phase could add capability without rewriting the entire system.

4. Overall Architecture and ROS 2 Data Flow

```
Gazebo world + ASV model
|
+--> odometry / IMU / camera / clock
|
|       |
|       +--> guidance --> heading / speed references
|
|               |
|               +--> PID or SMC --> differential thrust
|
+--> camera --> YOLO detector --> temporal confirmation --> mission FSM
|
|               |
|               +--> search / approach / arrival arbitration
|
Leader ASV --> leader_state --> cooperative coordinator --> follower reference --> follower
controller
```

4.1 Safety architecture

Safety was not treated as an afterthought. Watchdogs, timestamp checks, state-validity checks, command bounds, zero-thrust behavior and managed teardown were repeatedly tested. At mission completion the design required propulsion to be zero before the inspection image/logging action.

4.2 Scientific discipline

The project deliberately preserved failed trials and known limitations. For example, the static thermal-style camera scene is not a calibrated radiometric LWIR model, the disturbance plugin is not a full wave-spectrum or CFD model, and cooperative control was validated for two vehicles rather than a general fleet.

5. Phase 1 - Calm Ocean, Buoyancy and First Gazebo Scene

5.1 Objective

The first task was to prove the basic simulator stack: Gazebo Harmonic starts correctly, ROS 2 connects to it, a marine scene loads, simulation time is published, and a floating object can exist stably in the world.

5.2 Tasks completed

- Created the initial marine world and water scene.
- Established the `marine_description`, `marine_gazebo` and `marine_bringup` package structure.
- Validated Gazebo Harmonic GUI and headless execution under WSLg.
- Verified /clock integration for simulation time.
- Validated basic buoyancy behavior before introducing the catamaran dynamics.

5.3 Why this phase mattered

If buoyancy, clock, world loading or ROS-Gazebo bridging are unstable, every later metric becomes meaningless. The project therefore built the simulation from the bottom upward instead of immediately adding AI or complex control.

5.4 Result

Phase 1 passed and became the stable base scene used by later phases.

6. Phase 2 - Physical Catamaran, Thrusters, Odometry, IMU and TF

6.1 Objective

Phase 2 replaced the simple float test with an under-actuated catamaran USV capable of differential thrust and measurable motion.

6.2 Vehicle model

- Catamaran geometry with separate port and starboard propulsion.

- Buoyancy equilibrium around approximately 0.112 m flotation depth in the validated configuration.
- Hydrodynamic drag representation sufficient for the project's control experiments.
- Forward/reverse motion around ± 0.79 m/s in validation tests.
- Turning response around ± 0.39 rad/s in validation tests.

6.3 ROS interfaces added

- /asv/cmd_vel-style command interface feeding the propulsion mixer.
- Odometry publication.
- IMU publication.
- Pose / velocity information.
- TF tree for world/odom/base_link.
- Differential-thrust mixer converting surge/yaw request into port/starboard actuation.

6.4 What was learned

A marine controller cannot be validated purely against commanded velocities. The vehicle needs physical state feedback from simulated sensors and must respond through the same actuator path that later autonomous controllers will use.

7. Phase 3 - Cubic-Spline Path Generation and LOS Guidance

7.1 Objective

The next task was to create smooth paths and a guidance layer independent of the low-level controller.

7.2 Path types

- Straight path.
- Curved path.
- S-shaped path.

7.3 Cubic spline guidance

Natural cubic splines were used to produce smooth position and heading references. The guidance node generated reference-path information and LOS-style lookahead targets so the controller did not chase discrete waypoints directly.

7.4 Tests

The phase included 32 guidance tests and 4 mixer tests. A validation set of 662 samples was checked against odometry to confirm that the generated reference and measured vehicle data were aligned correctly.

7.5 Why LOS was selected

LOS guidance is well suited to path-following because it provides a lookahead heading target, reducing aggressive waypoint-to-waypoint steering. The controller therefore tracks a smooth desired heading while the guidance layer handles geometry.

8. Phase 4 - PID Path-Following Controller

8.1 Objective

Phase 4 closed the loop. The guidance layer produced a desired heading, and a PID controller generated yaw correction while maintaining forward speed.

8.2 Final PID gains

$K_p = 0.5$
 $K_i = 0.0$
 $K_d = 0.6$

8.3 Validation results

Path	CTE RMSE (m)	Heading RMSE (rad)	Completion time (s)
Straight	0.569	0.083	93.392
Curved	0.574	0.078	105.912
S-shaped	0.583	0.084	132.279

These results established the PID controller as the primary baseline. The important lesson is not that PID is universally optimal, but that it was demonstrably stable and repeatable in the validated calm-water operating conditions.

8.4 Tests

By the end of Phase 4, 63 tests were passing across guidance/control functionality.

9. Phase 5 - Camera, YOLO Perception, Search, Approach and Inspection Mission

9.1 Objective

Phase 5 transformed the project from path following into an autonomous mission. The vehicle had to search, detect a target vessel, confirm the detection over time, approach it, recover if the target disappeared, stop at a stand-off distance and perform inspection logging.

9.2 Camera and detector setup

The perception pipeline used a Gazebo camera and a frozen custom YOLOv5n6/P6 checkpoint. The checkpoint has four detection heads at strides 8/16/32/64, width multiplier 0.25, depth multiplier 0.33, approximately 3.11 million parameters, 10 classes and 640-pixel input size.

9.3 Dataset and class-mapping issue

The IRShips_YOLO archive contained 9000 grayscale infrared vessel images split into 7200 training, 900 validation and 900 test images. A critical issue was discovered: the class ordering in the archive YAML conflicted with the class names embedded in the checkpoint. Because the original training YAML was unavailable, the project intentionally avoided claiming reliable subclass-level performance. The mission therefore used generic vessel detection behavior.

9.4 Initial RGB failure

Initial Gazebo RGB, grayscale and CLAHE trials produced zero detections across 273 frames even when the model itself was functioning. This was an important negative result: a detector trained on infrared ship imagery does not automatically generalize to the rendered visual appearance of a standard Gazebo camera.

9.5 Thermal-style presentation approximation

A simulated thermal-style camera appearance was introduced by assigning object temperatures/intensities in the scene. This was explicitly documented as an approximation rather than calibrated radiometric long-wave infrared physics. Under that representation, 20/20 live frames produced vessel-overlap detections at confidence approximately 0.308-0.316 using confidence threshold 0.25 and IoU threshold 0.45.

9.6 Search behavior

A deterministic three-lane coverage search pattern was implemented. Two validation missions took about 205.5 s each, with CTE RMSE around 0.557 m, zero boundary deviation and safe zero-thrust behavior at completion.

9.7 Mission-state functionality

- Temporal detection confirmation to prevent a single noisy frame from triggering the mission.
- Arbitration between search guidance and visual target approach.
- Target bearing estimation from the camera/detection geometry.
- Target-loss handling and autonomous recovery back to search/reacquisition behavior.
- Independent depth/range-based stand-off arrival, rather than using Gazebo ground-truth distance for navigation.
- Inspection logging and saved image after propulsion was confirmed zero.

9.8 Integrated mission results before final hardening

The first integrated Phase 5E results achieved 3/6 full missions, with 1/2 of the final repeats passing. One successful arrival occurred at approximately 11.79 m before image capture after zero propulsion. These partial results were not hidden; they motivated later mission hardening.

9.9 Phase 5H final mission hardening

Phase 5H focused on reliability, confirmation, managed teardown and repeatability. In the final set of 10 new trials there were 8 inspections, 9 clean shutdowns and 7 combined end-to-end passes. Two trials timed out during arrival. One PID process produced a SIGSEGV after the inspection had

already completed and zero propulsion had been verified, so the report explicitly did not claim the segmentation-fault issue was completely eliminated.

9.10 Phase 5 testing scale

By the end of the mission phase, 305 automated tests plus the earlier Phase 1-4 live checks were passing. Zero propulsion at the validated end state was verified 10/10 in the final mission set.

10. Phase 6 - Sliding Mode Control (SMC)

10.1 Objective

Phase 6 added an alternative nonlinear controller without removing the PID baseline. The goal was to compare control characteristics under the same guidance architecture.

10.2 Frozen SMC configuration

```
lambda = 0.15
k = 0.05
phi = 0.08
omega = 0.5 s-1
```

The selected configuration was frozen in `src/marine_control/config/smc_phase6_checkpoint4_v1.yaml` while the original `smc_params.yaml` was preserved.

10.3 Control law

```
s = v_f - r + lambda * wrap(psi_f - psi)

tau* = 10 r + 20 |r| r + J0 [a_f + lambda (v_f-r) + k sat(s/phi)]

q = clip[-0.4, 0.4](h-1(tau*))
h(q) = 10q + 20|q|q
J0 = 45.993 kg m2
```

10.4 Stability claim

The project documented only conditional/local stability arguments. It did not claim a global sampled-data proof or a complete uncertainty-bound proof. This is an important scientific boundary.

10.5 Final PID-versus-SMC comparison

The final frozen comparison used nine matched pairs (three per path) from Batches 2-4. Batch 5 was planned but never run and therefore must not be described as experimental evidence. Across all nine pairs, PID produced lower CTE and tail-heading RMSE, while SMC produced lower yaw-request effort but greater command variation. Neither controller saturated in the selected runs.

10.6 Reliability

Across the final SMC/PID campaign, mission completion was 18/18, zero propulsion was 18/18 and process cleanliness was 17/18. The remaining process issue and the limitations of the diagnostic evidence were preserved in the final report.

11. Phase 7 - Deterministic Disturbance and Robustness Testing

11.1 Objective

Phase 7 tested the frozen controllers under reproducible external disturbances instead of changing the controller gains again.

11.2 Disturbance plugin

A minimal C++ Gazebo plugin, `marine::Disturbance`, applied deterministic world-frame force and yaw-moment profiles. The plugin was disabled by default. It was deliberately described as a disturbance injector, not as a physical ocean-wave spectrum, current CFD model or six-degree-of-freedom sea-state simulator.

11.3 Moderate disturbance profile

$A_x = 3 \text{ N}$
 $A_y = 8 \text{ N}$
 $A_z(\text{yaw moment}) = 1.5 \text{ N m}$

11.4 Six principal outcomes

Path / controller	Result	CTE RMSE (m)	Notes
Straight PID	COMPLETE	1.3293	Heading RMSE 0.1995 rad, 99.695 s
Straight SMC	FAIL	1.5797	220 s endpoint-acquisition timeout
Curved PID	FAIL	1.1494	220 s timeout
Curved SMC	FAIL	1.3914	220 s timeout
S-shaped PID	COMPLETE	1.2967	140.785 s
S-shaped SMC	FAIL	1.6176	220 s timeout

Only 2/6 principal missions completed under the chosen moderate disturbance profile. This is an example of the project preserving a scientifically useful negative result rather than tuning until everything passed.

11.5 Sensor diagnostics

- Latency and future-timestamp causal handling.
- Out-of-order data behavior.
- Safe-zero watchdog at 0.5 s.
- 0.7 s pause behavior.
- Explicit enable/recovery behavior.

11.6 Actuator diagnostics

- Symmetric authority reduction.
- Asymmetric port scaling.
- Delivered-thrust cap around ± 3 N in diagnostic tests.
- Safe zero and recovery.

11.7 Test scale

Phase 7 ended with 299 tests passing, including 16 disturbance-profile tests. Across all 11 phase-7 executions, zero propulsion and clean-process requirements were verified.

12. Phase 8 - Two-USV Cooperative Formation Control

12.1 Objective

Phase 8 extended the framework from one autonomous vessel to two. A leader followed the path and published cooperative state. A follower generated its own guidance reference from that state and maintained an 8 m in-trail formation.

12.2 Namespaces and state sharing

```
/leader/...  
/follower/...  
/cooperative/leader_state
```

Each vessel retained its own controller and propulsion path. Shared state was limited to the cooperative interface; the follower did not directly reuse the leader's actuator commands.

12.3 Formation definition

```
Desired body-frame offset: Delta x_b = -8.0 m, Delta y_b = 0.0 m  
Desired follower position:  $p_{F,des} = p_L + R(\psi_L) [-8, 0]^T$   
Desired follower heading:  $\psi_{F,des} = \psi_L$ 
```

12.4 Coordinator state machine

- INITIAL_TRACKING
- WAIT_FOR_TRACKING_THRESHOLD
- FORMATION_ACQUIRE
- FORMATION_HOLD
- COMMUNICATION_DEGRADED
- COMPLETE
- SAFE_STOP

12.5 Approved thresholds

- Leader CTE ≤ 1.0 m before cooperative gating.
- Follower CTE ≤ 1.5 m for tracking threshold.
- Formation acquisition timeout initially 45 s.
- Hold position error ≤ 1.0 m and $|\text{heading error}| \leq 0.2$ rad sustained for 1 s.

- State considered stale above 0.5 s age.
- Recovery window ≤ 3 s.
- SAFE_STOP after prolonged communication loss above 5 s.

12.6 Checkpoint B dual-vehicle smoke test

Leader and follower spawned separately, maintained independent odometry/TF/controller paths, and had exactly one cmd_vel publisher each. Minimum separation was about 4.987 m. Outputs were bounded and both vessels reached safe zero propulsion with clean child-process exit.

12.7 Checkpoint C state publisher and pure geometry

The leader cooperative state publisher used ROS odometry and guidance outputs only; Gazebo ground truth was forbidden from the control path. Published fields included time, sequence ID, x, y, heading, surge speed, yaw rate, path progress, CTE and a validity flag. Strict validation enforced numeric finiteness, causal timestamps and sequence monotonicity.

12.8 Original Checkpoint D failure

All three original principal formation missions failed acquisition. Root-cause analysis identified three different issues rather than blaming the controller generally:

- Straight: follower PID diagnostics were missing mandatory frame_id="odom", causing an invalid-input latch and effectively zero follower motion.
- Curved/S-shaped: direct atan2 bearing to a target behind the follower produced a near-reverse desired direction that was unsuitable for the forward-only vehicle.
- The acquisition timer started before the controller startup delay, consuming roughly 9.8 s of the 45 s window.

12.9 Minimal corrective design

The correction deliberately avoided retuning gains or relaxing thresholds. It added stamped diagnostics, measured acquisition time after startup, and replaced the direct bearing law with a Serret-Frenet-style reference: along-track error adjusted speed while lateral error adjusted heading.

```
u_cmd = clamp(u_L + K_v * e_long, 0, 0.6)
psi_des = wrap(psi_L - atan(e_lat / lookahead))
```

12.10 Completion-semantics correction

A later issue was conceptual rather than control-related. The original completion condition required leader and follower to enter the same endpoint sphere, which is incompatible with an 8 m in-trail formation. The semantic correction defined completion as leader mission complete plus follower holding a valid formation relative to the terminal leader state.

12.11 Corrective live results

Run	Result	Key observations
Straight corrective	Formation acquired	Acquired at 16.5 s sim / 6.5 s active; 92.92% hold; no losses; terminal semantics later

		corrected.
Curved corrective	PASS	Complete 115.55 s; terminal pos error 0.359 m; terminal heading error 0.015 rad.
S-shaped corrective	PASS	Complete 142.15 s; two formation losses at inflections and two autonomous reacquisitions; terminal pos error 0.966 m.

12.12 Communication-degradation diagnostics

- 0.20 s artificial latency: peak age 0.323 s, no false degradation, mission completed.
- 2 s dropout: HOLD -> COMMUNICATION_DEGRADED -> ACQUIRE -> HOLD -> COMPLETE, with degraded speed 0.3 m/s.
- 10 s dropout: communication-degraded state progressed to latched SAFE_STOP after >5 s; zero thrust verified.

12.13 Final Phase 8 status

Phase 8 finished with 515/515 tests passing. The cooperative framework was frozen with the two-vessel scope and communication-fault behavior documented.

13. Phase 9 - Final Integration, Traceability and Reproducibility

13.1 Objective

Phase 9 was not a new algorithm phase. It was the integration and evidence-sealing stage: prove the final workspace builds, run representative end-to-end scenarios, preserve history, and provide reproducible documentation.

13.2 Regression status

A clean build of all eight ROS 2 packages plus the C++ disturbance plugin passed, and the frozen automated suite reported 515/515 unit regressions with zero failures.

13.3 Final perception stress test

Condition	Continuity	Mean confidence / outcome
Nominal	100%	0.8496
Reduced contrast	100%	0.3479
Gaussian noise sigma=20	0%	Correctly remained in SEARCH
35% horizontal occlusion	100%	0.8211
0.6 s visual dropout	76%	Clean reacquisition

13.4 Final representative scenario A - navigation

Straight 60 m PID mission, seed 2026: COMPLETE at 93.3 s active / 103.35 s simulation time. CTE RMSE 0.574 m, heading RMSE 0.083 rad, endpoint CTE 0.0024 m. Nine of nine child processes exited cleanly and zero force was verified below 1e-6.

13.5 Scenario B - integrated perception mission

The final integrated mission demonstrated search -> YOLO detection -> confirmation -> target acquisition -> target loss -> reacquisition -> mission completion. Independent optical depth/range arrival occurred at 11.989 m rather than using bounding-box size as a proxy. The inspection image `actual_confirmed_target.png` was saved after zero propulsion, followed by a 12 s coast and clean teardown.

13.6 Scenario C - cooperative formation

Curved path, seed 2096, -8 m in-trail formation: formation hold was established at 16.50 s with position error 0.656 m and heading error 0.054 rad. There were no formation losses through the curve, and the mission completed at 115.55 s with dual zero propulsion and 18/18 clean child exits.

13.7 Final documentation artifacts

- `docs/requirements_traceability.md` - requirements REQ01-REQ19 all marked PASS.
- `docs/final_project_report.md` - full technical report.
- `docs/reproducibility.md` - reproducible launch/environment instructions.
- `docs/final_results_summary.md` - concise final metrics.
- `docs/phase9_capability_audit.md` - final capability audit.
- `docs/final_manifest.json` - artifact manifest.
- `README` and `scripts/run_marine_demo.sh` - final runnable interface.

13.8 Final frozen limitations

- Flat-ocean analytical physics remained the scientific base; nonlinear physical sea-state waves were not added to the validated controller model.
- Thrusters were direct-force models without cavitation or detailed motor time constants.
- Thermal-style imagery was not calibrated radiometric LWIR.
- Detector behavior was generic-vessel because subclass label mapping was unresolved.
- Cooperative control was validated for two vessels only.

14. Final Demo GUI, RViz, HUD and Wavy-Water Presentation Suite

14.1 Why a presentation layer was needed

The research framework originally ran mostly headless. That was suitable for automated experiments but not for a live academic demonstration. A final presentation-only layer was therefore added without changing the scientific algorithms.

14.2 GUI audit

- Major launch files already exposed `gui:=true/false`, but the wrapper forced headless operation.
- Reference paths and guidance markers existed as ROS topics but were not presented cleanly.
- The annotated perception image existed on `/asv/perception/annotated_image` but no dedicated demo viewer was wired.
- Cooperative metrics existed numerically but there were no spatial formation markers.

14.3 Presentation components

- `demo_visualizer.py`: downstream MarkerArray publisher for path, LOS, trajectory, target, stand-off ring and cooperative markers.
- `demo_camera_hud.py`: live OpenCV annotated-image HUD with mission state, confirmation count, range, stand-off target and bearing.
- `demo_telemetry_hud.py`: 5 Hz ASCII dashboard for controller, speed, heading, CTE, mission and formation state.
- Mode-specific RViz2 configurations for PID, mission and cooperative demos.
- `run_marine_demo.sh` flags `--gui` and `--rviz` for presentation-ready launches.

14.4 Water visual upgrade

The original water looked like a flat cyan plane. To improve demo realism without changing physics, a visual-only 3D ocean wave mesh and PBR normal-map material were introduced. The final mesh spans approximately 600 m x 600 m and uses multi-component static wave geometry. The visual peak-to-trough height was increased to about 0.24 m for clear presentation from a low oblique camera.

The critical distinction is that this wave mesh has no collision geometry and does not drive buoyancy or hydrodynamic forces. The validated physical model therefore remains calm-water analytical buoyancy/drag; the wave mesh is only a renderer asset.

14.5 GUI regression and recovery

An early attempt to customize Gazebo GUI plugins produced a white window. The root cause was a CameraTracking-related QML card expanding over the MinimalScene viewport when explicitly inserted into the world GUI block. Recovery restored a clean canonical gui block and let Gazebo fall back to its built-in `gui.config`. The horizon plane that caused Z-fighting with wave troughs was also removed.

14.6 Final presentation cleanup

- Gazebo grid hidden.
- EntityTree and ComponentInspector collapsed by default.
- Camera moved closer to a low quarter-stern view.
- Water roughness/specular parameters softened to avoid blown-out white glare.
- Full viewport preserved while retaining controls and world statistics.

15. Build and Runtime Problems Solved During Demo Preparation

15.1 Missing `marine_disturbance` setup file

Sourcing `install/setup.bash` produced a missing `local_setup.bash` error for `marine_disturbance`. Investigation showed that `colcon` was discovering an archived evidence copy of the package under `docs/evidence/phase5/stage_bc/regressions/src/marine_disturbance` instead of the real package at `src/marine_disturbance`.

15.2 Windows Anaconda protobuf contamination

The failed build mixed Ubuntu protobuf headers with Windows Anaconda headers mounted under /mnt/c/Users/LENOVO/anaconda3/Library/include. That produced errors such as PROTOBUF_NAMESPACE_OPEN missing, MapEntry template-parameter conflicts, TaggedPtr errors and incomplete gz::msgs types. The lesson is that WSL build environments can accidentally inherit Windows development paths through PATH/CPATH-style variables.

15.3 Correct package source

The real package existed at src/marine_disturbance and a validated Phase-7 overlay already existed under install-phase7. For demo execution the Phase-7 overlay was sourced instead of rebuilding archived evidence.

15.4 Gazebo GUI crash

The Gazebo server successfully created world topics, thruster interfaces, odometry and scene services, but the GUI process exited with code -11 under the native renderer. Setting LIBGL_ALWAYS_SOFTWARE=1 was used as the stable presentation fallback. The important diagnostic lesson was to separate simulation-server health from rendering-process health.

16. Current Commands for Running All Simulation Modes

16.1 Environment preparation

```
cd /home/sireesha/marine_ws
source /opt/ros/humble/setup.bash
source install/local_setup.bash 2>/dev/null || true
source install-phase7/local_setup.bash
export LIBGL_ALWAYS_SOFTWARE=1
```

16.2 Demo commands

```
./scripts/run_marine_demo.sh basic --gui
./scripts/run_marine_demo.sh pid --gui --rviz
./scripts/run_marine_demo.sh smc --gui --rviz
./scripts/run_marine_demo.sh disturbance --gui --rviz
./scripts/run_marine_demo.sh perception --gui --rviz
./scripts/run_marine_demo.sh mission --gui --rviz
./scripts/run_marine_demo.sh cooperative --gui --rviz
```

16.3 Recommended viva sequence

1. PID navigation - prove guidance + closed-loop control.
2. Autonomous mission - prove perception + mission logic + safe arrival.
3. Cooperative formation - prove multi-vehicle coordination and communication handling.

16.4 Clean shutdown

Use Ctrl+C in the launcher terminal and allow the managed wrapper to stop auxiliary visualizers, ROS nodes and Gazebo processes. Avoid killing random child processes during a formal demo because that bypasses the validated teardown path.

17. Future Real Gimbal-Camera Integration

The current architecture is compatible with a real stabilized gimbal camera. The clean design is to keep perception and mission logic unchanged and replace the simulated camera interface with a hardware driver node.

```
Real gimbal camera -> driver/SDK node -> /gimbal/image_raw -> detector
                                     -> /gimbal/state -> TF
Mission / tracking node -> /gimbal/cmd_pan_tilt -> gimbal hardware
```

17.1 Recommended ROS 2 interfaces

- /gimbal/image_raw
- /gimbal/camera_info
- /gimbal/pan
- /gimbal/tilt
- /gimbal/cmd_pan
- /gimbal/cmd_tilt
- /gimbal/status

17.2 TF chain

```
base_link -> gimbal_base -> gimbal_yaw -> gimbal_pitch -> camera_link
```

The exact implementation depends on the eventual hardware interface: USB, Ethernet/RTSP, serial, CAN, MAVLink, ONVIF or a vendor SDK. This has not yet been implemented and should be treated as a future hardware-integration phase.

18. Key Limitations and Scientific Claims You Should and Should Not Make

18.1 Claims supported by the project

- The framework integrates ROS 2 Humble with Gazebo Harmonic for autonomous marine simulation.
- The ASV follows spline paths using LOS guidance with selectable PID/SMC control.
- The frozen YOLO model can be integrated into a simulated thermal-style vessel-detection mission.
- The mission stack supports search, confirmation, loss/reacquisition, approach, range-based arrival and safe inspection.
- A deterministic disturbance plugin enables repeatable controller robustness experiments.
- Two ASVs can perform in-trail cooperative formation with latency/dropout handling in the validated scenarios.
- The system includes regression tests, evidence capture, metrics and managed teardown.

18.2 Claims you should avoid

- Do not call the thermal-style scene a physically calibrated thermal imager.

- Do not claim reliable individual vessel-subclass accuracy because class mapping remained unresolved.
- Do not claim the visual wave mesh is a physical sea-state model.
- Do not claim SMC was superior to PID in the final matched comparison; PID had lower CTE/tail-heading RMSE in all nine matched pairs.
- Do not claim a global mathematical stability proof for the SMC.
- Do not claim robustness under arbitrary disturbances or real ocean conditions.
- Do not claim fleet-scale cooperation beyond the two-vessel experiments.

19. From-Scratch Study Plan

Use this sequence if you want to relearn the project from zero rather than memorizing the final commands.

Stage	What to study	Practical exercise
1	Linux + WSL2 basics	Navigate workspace, source ROS environment, inspect processes.
2	ROS 2 nodes/topics/services/parameters	Use ros2 node list, topic list, topic echo and topic info.
3	TF and coordinate frames	Draw world/odom/base_link/camera frames and inspect TF.
4	Gazebo Harmonic	Load a world, inspect entities, understand server vs GUI.
5	Marine vehicle physics	Explain buoyancy, drag, differential thrust and under-actuation.
6	Path geometry	Generate straight/curved/Spline and compute path tangent.
7	LOS guidance	Explain lookahead target and cross-track correction.
8	PID	Derive P/I/D terms and interpret $K_p=0.5$, $K_i=0$, $K_d=0.6$.
9	SMC	Understand sliding surface, saturation boundary layer and yaw inversion.
10	Computer vision	Understand YOLO input, confidence, IoU and temporal confirmation.
11	Mission FSM	Draw SEARCH -> ACQUIRE -> APPROACH -> ARRIVAL/INSPECT with loss recovery.

12	Robustness	Explain deterministic disturbance, latency, stale-data and actuator-cap tests.
13	Cooperation	Derive the -8 m body-frame formation transformation.
14	Experiment design	Understand seeds, matched trials, failure preservation and metrics.
15	Demo	Run PID, mission and cooperative modes and narrate the data flow.

20. Viva Questions and Short Answers

Q: Why ROS 2?

A: It provides modular communication, message typing, namespaces, TF, parameterization and tooling needed to separate sensing, guidance, control, mission logic and cooperation.

Q: Why Gazebo Harmonic?

A: It provides modern Gazebo Sim physics/rendering and integrates with ROS 2 through `ros_gz_bridge`. The project used Gazebo Sim 8.x.

Q: Why separate guidance and control?

A: Guidance solves path geometry and desired heading; control solves actuator effort. Separation makes each layer testable and controllers interchangeable.

Q: Why LOS guidance?

A: A lookahead target produces smoother path following than steering directly at discrete waypoints.

Q: Why was Ki zero in PID?

A: The validated tuning did not require integral action; adding unnecessary integral action could introduce windup and overshoot.

Q: What does differential thrust do?

A: Equal thrust mainly produces surge; a port/starboard difference creates yaw moment.

Q: Why did RGB detection fail?

A: The custom checkpoint came from an infrared ship dataset, and the Gazebo RGB appearance had a strong domain mismatch.

Q: Is your thermal camera realistic?

A: It is a controlled thermal-style approximation for perception integration, not a calibrated radiometric LWIR simulation.

Q: Why temporal confirmation?

A: To prevent one-frame false detections from changing mission behavior.

Q: How is arrival detected?

A: Using an independent optical depth/range-based estimate with a 12 m stand-off target, not Gazebo ground truth.

Q: What happens on target loss?

A: The mission controller leaves visual approach and executes recovery/search logic until the target is reacquired or safety logic intervenes.

Q: Why add SMC?

A: To evaluate a nonlinear robust-control alternative under the same guidance and vehicle model.

Q: Was SMC better than PID?

A: Not in the final matched accuracy results. PID had lower CTE and tail-heading RMSE in all nine matched pairs; SMC used lower yaw-request effort but had greater command variation.

Q: What did Phase 7 prove?

A: It provided reproducible external-load and fault diagnostics. It did not prove arbitrary real-sea robustness.

Q: How is formation defined?

A: Follower target is 8 m behind the leader in the leader body frame with matching desired heading.

Q: Why was the original cooperative completion condition wrong?

A: Both vessels cannot occupy the same endpoint sphere while maintaining an 8 m in-trail formation. Completion semantics had to respect the formation geometry.

Q: What happens during communication dropout?

A: Short dropouts enter a degraded state and can recover; prolonged loss beyond the safety threshold leads to SAFE_STOP and zero propulsion.

Q: Why keep failed trials?

A: They are part of the scientific evidence and reveal operating limits and implementation defects.

Q: Are the new visible waves part of vehicle physics?

A: No. They are presentation-only static 3D wave geometry with PBR shading; validated buoyancy and drag remain the analytical calm-water model.

Q: Can the project use a real gimbal camera?

A: Yes, through a ROS 2 hardware driver publishing image/state topics and accepting pan/tilt commands, but the exact interface depends on the hardware model.

Appendix A - Important Parameters and Results

Parameter	Frozen / reported value
PID gains	Kp 0.5, Ki 0, Kd 0.6
SMC	lambda 0.15, k 0.05, phi 0.08, omega 0.5 s ⁻¹
SMC inertia parameter	J0 = 45.993 kg m ²
SMC normalized command limit	q in [-0.4, 0.4]
ASV nominal forward speed reference	0.6 m/s in principal path experiments
Cooperative formation	-8 m longitudinal, 0 m lateral
Coop stale threshold	0.5 s
Coop prolonged dropout SAFE_STOP	>5 s
Mission stand-off	12 m
YOLO input	640
YOLO parameters	~3.11 M
YOLO heads	strides 8/16/32/64
Dataset size	9000 infrared vessel images
Final regression suite	515/515 tests
Final PID scenario CTE RMSE	0.574 m
Final mission arrival	11.989 m
Final cooperative completion	115.55 s curved scenario

Appendix B - Important ROS 2 Topics and Frames

Exact topic names can vary by launch mode and namespace; the following are the important interfaces repeatedly used by the completed framework.

Purpose	Representative interface
Simulation time	/clock
ASV odometry	/asv/odom or bridged model odometry
IMU	/asv/imu
Guidance reference path	/asv/guidance/reference_path
Guidance markers	/asv/guidance/markers
Annotated perception image	/asv/perception/annotated_image
Mission diagnostics	/asv/mission/diagnostics
Leader cooperative state	/cooperative/leader_state
Presentation markers	/marine/visualization_markers

Leader namespace	/leader/...
Follower namespace	/follower/...

Typical frame logic:
world/odom -> base_link -> camera_link

Cooperative:
world/odom -> leader/base_link
world/odom -> follower/base_link

Appendix C - Troubleshooting Checklist

4. Confirm you are in /home/sireesha/marine_ws, not the old maritime_ws.
5. Source /opt/ros/humble/setup.bash first.
6. If install/setup.bash reports marine_disturbance missing, use the validated install-phase7 overlay and inspect colcon package discovery before rebuilding.
7. If colcon finds docs/evidence/.../marine_disturbance, do not build that archived copy.
8. If protobuf errors mention both /usr/include and /mnt/c/Users/.../anaconda3, remove Windows Anaconda paths from the WSL build environment.
9. If Gazebo server runs but GUI exits -11, treat it as a renderer problem rather than a ROS/controller failure.
10. Use LIBGL_ALWAYS_SOFTWARE=1 as the known stable GUI fallback for the current demo configuration.
11. If HUD stays at Sim Time 0.00, verify /clock and check whether Gazebo shut down immediately after a GUI failure.
12. For PID mode, absence of a thermal image is not a controller failure; use the PID-specific RViz layout.
13. Use managed Ctrl+C teardown and verify no orphaned gz/ros2/marine processes remain before starting the next demo.

Appendix D - Glossary

Term	Meaning
ASV	Autonomous Surface Vehicle.
CTE	Cross-track error; perpendicular distance from reference path.
LOS	Line-of-Sight guidance using a lookahead point.
PID	Proportional-Integral-Derivative controller.
SMC	Sliding Mode Control.
FSM	Finite State Machine.
TF	ROS transform system for coordinate frames.
YOLO	You Only Look Once object detector family.
IoU	Intersection over Union used in detection overlap/NMS.
RMSE	Root Mean Square Error.
PBR	Physically Based Rendering; used only for

	visual water appearance here.
WSLg	Windows Subsystem for Linux GUI integration.
Stand-off distance	Safe target-relative distance maintained before inspection.
Managed teardown	Controlled shutdown sequence that forces safe outputs and terminates owned processes cleanly.

Final Learning Advice

Do not try to memorize the whole project at once. Rebuild the mental model layer by layer: simulator -> vehicle -> sensors -> guidance -> controller -> perception -> mission -> robustness -> cooperation -> evidence. If you can explain the data flow between these layers and reproduce the three main demos, you understand the project much more deeply than someone who only remembers commands.

The most important research habit demonstrated by this project is disciplined scope control: freeze validated components, isolate one new capability at a time, preserve failures, measure before retuning, and distinguish presentation improvements from scientific model changes.